

Docker on AWS

Moderate & Advanced Labs on an EC2 Instance

Networking, resource limits, ECR, multi-stage builds, Docker Swarm, and a reverse-proxied multi-service stack — all step by step.

Before You Start

These labs assume you already have an EC2 instance running with Docker installed (Amazon Linux 2023, t2.micro/t3.micro or larger), and that you can connect via EC2 Instance Connect or SSH. If not, revisit the setup steps from the earlier Docker introduction guide first.

Recommended instance size: Labs 8-10 (advanced) are lighter on a t3.small or larger, since Swarm and multiple containers running together can strain a t2.micro's 1 GB RAM.

For Labs 5-7, also make sure the EC2 security group allows inbound TCP on ports 8080-8090 for testing (in addition to 22 and 80 from earlier).

Moderate Labs

Lab 5 — Custom Docker Networks & Container-to-Container Communication

MODERATE

Goal: connect two containers on a private Docker network and have them talk to each other by container name — without Docker Compose.

1. Create a user-defined bridge network:

```
docker network create app-net
```

2. Run a Redis container attached to that network:

```
docker run -d --name redis-cache --network app-net redis:alpine
```

3. Run a second container on the same network and ping the first by name:

```
docker run --rm --network app-net busybox ping -c 3 redis-cache
```

This works because Docker's embedded DNS resolves container names to their internal IPs automatically on a user-defined network — the default bridge network does not offer this.

4. Inspect the network to see both containers attached:

```
docker network inspect app-net
```

5. Clean up:

```
docker rm -f redis-cache  
docker network rm app-net
```

Lab 6 — Resource Limits, Health Checks & Restart Policies

MODERATE

Goal: constrain a container's CPU/memory, add a health check, and make it self-heal automatically.

6. Run a container with hard memory and CPU limits:

```
docker run -d --name limited-app --memory="128m" --cpus="0.5" nginx:alpine
```

7. Confirm the limits are applied:

```
docker inspect limited-app --format='{{.HostConfig.Memory}}  
{{.HostConfig.NanoCpus}}'
```

8. Watch live resource usage:

```
docker stats limited-app
```

9. Add a health check and an auto-restart policy (recreate the container with both):

```
docker rm -f limited-app  
docker run -d --name limited-app \  
  --memory="128m" --cpus="0.5" \  
  --restart unless-stopped \  
  --health-cmd="curl -f http://localhost/ || exit 1" \  
  --health-interval=30s --health-timeout=5s --health-retries=3
```

```
--health-interval=15s --health-retries=3 \
nginx:alpine
```

10. Check the health status:

```
docker ps          # STATUS column will show (healthy) after ~15-30s
```

With `--restart unless-stopped`, if the container process crashes or the EC2 instance reboots, Docker restarts it automatically — a basic form of self-healing without an orchestrator.

Lab 7 — Push a Custom Image to Amazon ECR

MODERATE

Goal: publish the custom image from Lab 2 (or any local image) to a private Amazon ECR registry, the AWS-native alternative to Docker Hub.

11. In the AWS Console, go to ECR → Repositories → Create repository. Name it `my-webapp`, keep it private, and create it.
12. Copy the repository URI shown, e.g. `123456789012.dkr.ecr.us-east-1.amazonaws.com/my-webapp`.
13. On the EC2 instance, install the AWS CLI if not already present:

```
sudo yum install -y aws-cli
```

14. Authenticate Docker to ECR (replace region and account ID):

```
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-
1.amazonaws.com
```

15. Tag your local image to match the ECR repository URI:

```
docker tag my-webapp:v1 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-
webapp:v1
```

16. Push it:

```
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-webapp:v1
```

17. Confirm it landed in the console under your ECR repository, or via:

```
aws ecr list-images --repository-name my-webapp
```

The EC2 instance's IAM role (or your configured credentials) needs at minimum the `AmazonEC2ContainerRegistryPowerUser` policy for this to succeed.

Advanced Labs

Lab 8 — Multi-Stage Builds to Shrink Image Size

ADVANCED

Goal: use a multi-stage Dockerfile so build tools never ship in your final image — dramatically reducing size and attack surface.

18. Create a small Go-style example (works conceptually the same for Node/Python too):

```
mkdir ~/multistage-lab && cd ~/multistage-lab

cat > app.py << 'EOF'
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello from a multi-stage image!')
HTTPServer(('0.0.0.0', 8000), H).serve_forever()
EOF
```

19. Write a multi-stage Dockerfile:

```
cat > Dockerfile << 'EOF'
# Stage 1: build/prepare (would include compilers, package managers, etc.)
FROM python:3.12 AS builder
WORKDIR /app
COPY app.py .

# Stage 2: minimal runtime image only
FROM python:3.12-alpine
WORKDIR /app
COPY --from=builder /app/app.py .
EXPOSE 8000
CMD ["python3", "app.py"]
EOF
```

20. Build and compare sizes against a single-stage equivalent:

```
docker build -t multistage-app:slim .
docker images | grep -E 'multistage-app|python'
```

The final image only contains the lightweight python:3.12-alpine layer plus your script — none of the larger build-stage layers are included, since only files explicitly copied with `--from=builder` carry over.

21. Run and test it:

```
docker run -d --name multistage-test -p 8082:8000 multistage-app:slim
curl http://localhost:8082
```

Lab 9 — Docker Swarm: Scaling and Rolling Updates

ADVANCED

Goal: turn your single EC2 instance into a one-node Swarm cluster, run a replicated service, scale it, and perform a zero-downtime rolling update.

22. Initialize Swarm mode on the instance:

```
docker swarm init --advertise-addr $(curl -s  
http://169.254.169.254/latest/meta-data/local-ipv4)
```

This uses the EC2 instance metadata service to grab its private IP automatically for Swarm's internal advertise address.

23. Deploy a replicated Nginx service with 3 replicas:

```
docker service create --name web-service --replicas 3 -p 8083:80 nginx:alpine
```

24. Check the service and its tasks:

```
docker service ls  
docker service ps web-service
```

25. Scale it up to 5 replicas on the fly:

```
docker service scale web-service=5
```

26. Perform a rolling update to a new image tag with zero downtime:

```
docker service update \  
  --image nginx:1.27-alpine \  
  --update-parallelism 1 \  
  --update-delay 10s \  
  web-service
```

Swarm replaces one replica at a time, waiting 10 seconds between each, so the service keeps serving traffic throughout the update.

27. Clean up when finished:

```
docker service rm web-service  
docker swarm leave --force
```

Note: a true multi-node Swarm would span several EC2 instances (one manager, several workers) joined with `docker swarm join` — this lab uses a single-node Swarm to demonstrate the orchestration commands without provisioning extra instances.

Lab 10 — Reverse Proxy in Front of Multiple App Containers

ADVANCED

Goal: run two independent app containers behind a single Nginx reverse proxy container, routing by path — a pattern common in real production setups.

28. Create a project folder:

```
mkdir ~/reverseproxy-lab && cd ~/reverseproxy-lab
```

29. Create two tiny backend apps (reuse busybox httpd for simplicity):

```
mkdir app1 app2  
echo '<h1>App 1</h1>' > app1/index.html  
echo '<h1>App 2</h1>' > app2/index.html
```

30. Create the Nginx reverse proxy config:

```
cat > nginx.conf << 'EOF'
events {}
http {
    server {
        listen 80;
        location /app1/ {
            proxy_pass http://app1:80/;
        }
        location /app2/ {
            proxy_pass http://app2:80/;
        }
    }
}
EOF
```

31. Create a shared network and start both backend containers:

```
docker network create proxy-net
docker run -d --name app1 --network proxy-net -v
$(pwd)/app1:/usr/share/nginx/html nginx:alpine
docker run -d --name app2 --network proxy-net -v
$(pwd)/app2:/usr/share/nginx/html nginx:alpine
```

32. Start the reverse proxy container, mounting your config:

```
docker run -d --name proxy --network proxy-net -p 8084:80 \
-v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \
nginx:alpine
```

33. Test both routes through the single proxy port:

```
curl http://localhost:8084/app1/
curl http://localhost:8084/app2/
```

Each request enters through the same container and port, and the proxy routes it internally by path to the correct backend — the same architecture pattern used by ALB + target groups or Kubernetes Ingress, just implemented directly with Docker networking.

34. Clean up:

```
docker rm -f proxy app1 app2
docker network rm proxy-net
```

Clean-Up Checklist

- Stop all remaining containers: `docker stop $(docker ps -q)`
- Remove dangling images/volumes if needed: `docker system prune -a --volumes`
- Leave Swarm mode if still active: `docker swarm leave --force`
- Terminate the EC2 instance from the console if you're fully done, to stop billing.

Summary

These labs move from single-host networking and resource governance (Labs 5-6), to publishing images through a real AWS registry (Lab 7), and finally into production-adjacent patterns: lean multi-stage images, container orchestration with Swarm, and a reverse-proxied multi-service architecture (Labs 8-10). Together with the introductory labs, this covers the practical range of Docker skills used in real AWS deployments — from a single container up to a small orchestrated, registry-backed stack.